

Class 5 Notes

Email Extraction at Scale — designing around a 100:1 capacity gap with triage, priority queues, and a two-tier extractor

1. Problem Statement

Many emails carry structured data in text form: flight and hotel confirmations, electronic bills and receipts, package delivery notifications. Our team has built an extraction system that converts such emails into actual structured data, consumed by client products like Assistant, Calendar, and Wallet (e.g. a flight confirmation automatically becomes a calendar entry).

The catch: the extractors are ML-heavy and CPU-expensive — they can process only ~1,000 emails/second. Email arrives at ~100,000 emails/second. That is a 100× gap between demand and capacity. Design a system that minimizes the latency with which IMPORTANT emails are extracted and indexed.

Read the problem statement like an engineer

The problem does NOT say “extract every email fast” — it says minimize latency for IMPORTANT emails. That single word is the entire design: since we physically cannot process everything, the system's real job is deciding what deserves the scarce extractor capacity, and in what order. When demand exceeds supply by 100×, prioritization IS the architecture.

2. Requirements

2.1 Functional Requirements

1. **FR1:** Classify every incoming email as extraction-worthy or not, using cheap signals (sender, tags, subject, metadata, preview). Assumption confirmed in class: sender is a reliable base signal for importance.
2. **FR2:** Assign each extraction-worthy email a priority: P0 (product must act immediately — e.g. flight change), P1 (calendar-worthy — meetings, bookings), P2 (bills, receipts, deliveries). Promotional/social → ignore.
3. **FR3:** Extract structured fields from email HTML and index them for client products.
4. **FR4:** Publish extracted structured data to Kafka topics (organized by data type) so any downstream service — Assistant, Calendar, Wallet, search index — can subscribe and consume independently.

2.2 Non-Functional Requirements

Attribute	Target	Design consequence
Throughput in	~100,000 emails/sec sustained	Triage stage must be cheap and horizontally scalable

Attribute	Target	Design consequence
Extractor capacity	~1,000 emails/sec (heavy ML)	The scarce resource — everything protects it
Latency SLOs	P0: seconds · P1: ≤ 5 min · P2: ≤ 1 hour	Separate queues + dedicated/weighted workers per tier
Consistency	Eventual — an extraction landing minutes later is fine (except P0)	Queues, batching, and buffering are all legal
Durability	No acknowledged email may be silently lost from the pipeline	Persistent queues, retries, dead-letter queue
Accuracy	Wrong extractions are worse than slow ones (Calendar shows wrong flight time!)	Validation stage + confidence thresholds

3. The Core Insight: Close the 100× Gap in Two 10× Steps

You cannot buy your way out of a 100× capacity gap — you must shrink the work. The class solution does it with two independent 10× reductions, each exploiting a different property of email:

5. **~90% of email doesn't deserve extraction (triage).** Promotions, social updates, newsletters — cheap signals identify them without ever parsing the body. 100K/s → 10K/s.
6. **~90% of the remainder doesn't need ML (template cache).** Companies send templated email; the layout of an IndiGo confirmation barely ever changes. If we've seen the template before, we already KNOW where the fields live (their XPaths) — extraction becomes a cheap DOM lookup. Only genuinely new templates hit the expensive ML extractor. 10K/s → ~1K/s of heavy work — exactly our capacity.

Notice the shape of the argument: measure the workload, find the skew (most email is junk; most templates repeat), and put a cheap filter in front of the expensive resource at each level of skew. This is the funnel pattern from Class 4's write-path taming, applied to compute instead of writes.

4. Stage 1 — Cheap Triage and Priority Classification

The triage service must sustain 100K/s, so it may only use signals that are available without rendering or deeply parsing the HTML body:

Signal	What it tells us	Example rules
Sender domain + reputation	Strongest importance signal (class assumption)	airlines/banks → extract; known bulk senders → ignore
Existing tags/labels	Mail pipeline already classifies broad type	Promotional, Social → ignore immediately
Subject line	Cheap keyword/pattern hit	"itinerary", "invoice", "OTP", "out for delivery"
Preview snippet (first	Confirms subject signal without	"Your flight PNR..." → P0/P1 candidate

Signal	What it tells us	Example rules
~100 chars)	full parse	
Metadata: sending IP, SPF/DKIM, attachments	Authenticity + type hints	unauthenticated bulk IP → ignore; PDF invoice attached → P2

Implementation notes:

- Rules first, then a lightweight ML model (logistic regression / small gradient-boosted tree over these features) for the ambiguous middle — NOT the heavy extractor model. Triage cost per email must be microseconds.
- The service is stateless (reputation/rule tables cached in memory, refreshed periodically) → scale horizontally to any QPS.
- Priority mapping from class: P0 = a product must act now (flight change, OTP-style security, same-day travel); P1 = calendar-worthy within minutes (meetings, bookings); P2 = everything else worth extracting (bills, receipts, shipment updates).
- Tune the classifier for high recall on importance: a false 'ignore' on a flight email is a product failure, while a false 'extract' on a newsletter merely wastes a little P2 capacity. When the costs of the two errors differ, bias the threshold toward the cheap error — and let the ignored 90% still get normal mail indexing, so nothing is lost, only not extracted.

5. Stage 2 — Priority Queues and SLA-Aware Scheduling

- One queue per priority (three Kafka topics), not one clever priority queue: independent lag monitoring, independent scaling, and no risk of P2 floods burying P0 messages.
- Messages carry the email ID and triage metadata — never the body. Bodies stay in mail storage; workers fetch on demand. Keeps queues small: even P2's worst case ($7.5K/s \times 3,600s \approx 27M$ buffered messages $\times \sim 200$ bytes ≈ 5.4 GB) is trivial for Kafka.
- Scheduling: a dedicated worker pool consumes P0 (its SLA is seconds — it can never wait behind anything), while a weighted pool serves P1 heavily and P2 lightly. P2's one-hour SLA is not a weakness — it is the system's shock absorber: during a spike, P2 backlog grows and drains later, protecting P0/P1 latency.
- Backpressure and safety: consume-side autoscaling on queue depth AND oldest-message age (depth alone hides slow drain); retries with exponential backoff; after N failures a message goes to a dead-letter queue for offline inspection — one malformed email must never wedge a partition.
- Idempotency: workers may process a message twice (at-least-once delivery), so extraction writes are keyed by email ID — replays overwrite harmlessly. Same discipline as Class 2's readings dedupe key.

6. Stage 3 — The Two-Tier Extractor (the Deep Dive)

In the architecture, the extraction service is a black box: priority topics in, structured data out. But the class deep-dive into what's inside that box is where the second 10× comes from — so we record it here:

7. **Understand the expensive step.** An email body is HTML. The ML extractor's real output is structural knowledge: the XPath's where the fields live — 'departure time is at /html/body/table[2]/tr[3]/td[1]'. Deriving that mapping is what burns CPU.
8. **Notice what's stable.** Templates are per-company assets that change rarely — every IndiGo confirmation this year has the same skeleton. The expensive output is therefore massively reusable.
9. **So cache the computation, not the data.** Fingerprint each email — hash(sender domain + DOM structure) — and look up an XPath cache (Redis). Hit (~90%): apply cached XPath's with a plain DOM parser — cheap, no ML. Miss: run the ML extractor, then write the derived XPath's back to the cache so the next thousand emails from that template take the fast path.
10. **Validate everything.** Every result (both paths) passes schema and confidence checks — dates parse, prices are numeric, required fields present. A failing fast-path result means the company silently redesigned its template: invalidate the cache entry and re-run through ML, which re-learns and re-populates. The cache heals itself.

The pattern, named

This is Class 4's 'precompute at write time' wearing a new outfit: amortize an expensive computation across all future inputs that share structure. You'll see the same move in query planners (cache compiled plans), JIT compilers (compile hot paths once), and regex engines (compile the pattern, run it cheaply). Whenever a heavy computation's OUTPUT depends on something more stable than the input — here, the template rather than the email — cache at the stable level.

7. The Output Side — Structured Data as Kafka Topics

Extraction results are not pushed to specific clients; they are published to per-type Kafka topics — flights/hotels, bills/receipts, package deliveries. Downstream services simply subscribe to what they need. Why this is the right shape:

- **Decoupling:** the pipeline doesn't know or care who consumes. Calendar reads flights; Wallet reads bills; a brand-new product next year just subscribes — zero changes upstream.
- **Independent pace:** each consumer processes at its own speed with its own consumer group and lag; a slow consumer never back-pressures extraction or other consumers.
- **Replay:** Kafka retention means a new or rebuilt consumer can rewind and re-read recent history — no custom backfill API needed for onboarding.
- **Idempotency:** messages are keyed by user + email ID, so re-extraction or consumer retries overwrite/dedupe cleanly (and log compaction can keep only the latest version per email).

8. Does the Math Close? (Always Check)

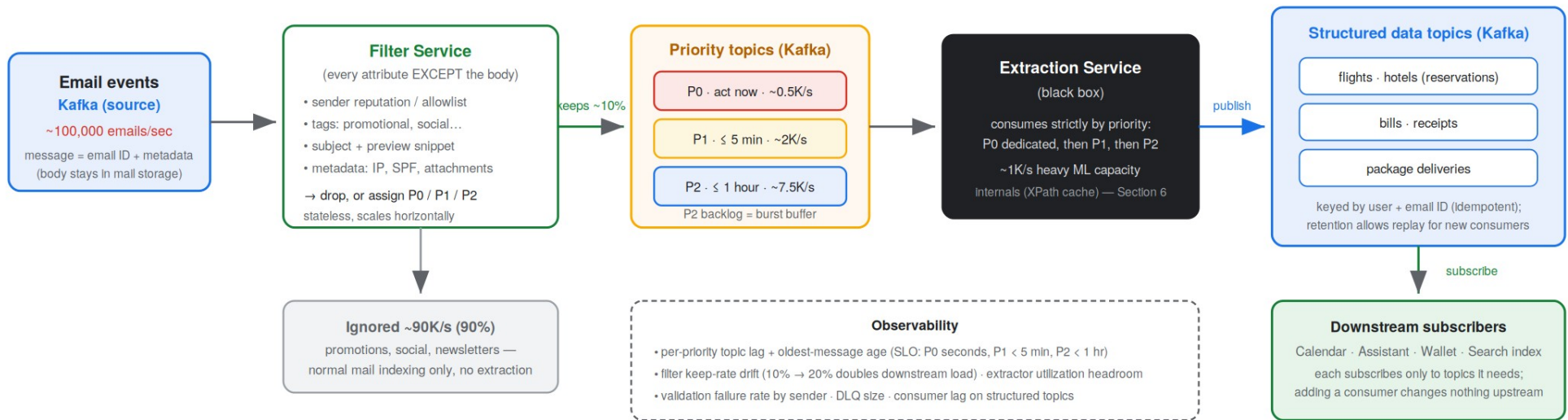
Stage	Rate	Cost per email	Check
Triage	100K/s	microseconds (rules + tiny model)	Horizontal scale; no body parsing

Stage	Rate	Cost per email	Check
Fast-path extraction	~9K/s (90% of 10K)	cheap DOM parse + XPath apply	Modest CPU fleet handles this
ML extraction	~1K/s (10% of 10K)	heavy ML	= stated extractor capacity
Headroom warning	—	—	Running the ML tier at 100% utilization means any burst blows latency. Provision ~30–50% headroom or accept that P2 backlog absorbs the difference — say this in the interview.

9. Architecture Diagram

Email Extraction Pipeline — Kafka In, Filter, Extract, Kafka Out

100K/s in → filter keeps ~10% and assigns priority → extractor (black box, ~1K/s heavy capacity) → structured data topics → any subscriber



10. Metrics, Monitoring & Alarms

Metric	Why it matters	Alarm when...
Per-priority queue depth + oldest-message age	Direct view of SLA health; age catches slow drain that depth hides	P0 age > 10 s; P1 age > 3 min; P2 age > 45 min
XPath cache hit rate	THE load-bearing number — if 90% drops to 70%, ML demand triples to 3K/s and the design collapses	hit rate < 85%
Triage keep-rate	Drift detector: if 10% kept becomes 20%, downstream load doubles silently	keep-rate deviates > 30% from baseline
Validation failure rate (by sender)	Spike for one sender = template redesign; global spike = extractor bug	per-sender failures > 5%
ML extractor utilization	The scarce resource; must keep headroom	> 80% sustained for 15 min
DLQ size	Poison messages / systematic parse failures	> threshold or growing
End-to-end extraction latency by priority	The actual product promise (p99 per tier)	SLO burn rate on any tier

11. Failure Modes Worth Naming in the Interview

- **Silent template redesign:** the scariest failure — cached XPaths still 'work' but grab the wrong cells, and Calendar shows a wrong flight time. Defense in depth: schema/confidence validation, per-sender failure-rate alarms, and a continuous audit that samples ~0.1% of fast-path results through the ML path and compares answers.
- **P0 storm:** a fog day cancels 500 flights and every airline emails everyone at once. Dedicated P0 pool autoscale + P0 has its own headroom budget; worst case, P1/P2 workers are borrowed (their SLAs have slack by design).
- **Cache cold start / Redis loss:** everything becomes a miss → 10K/s hits a 1K/s ML tier. Mitigations: persist the XPath store durably (Redis is a cache over a real table), and on cold start let the P2 backlog absorb while the cache re-warms from the most frequent senders first.
- **Triage false negatives:** an important sender gets ignored. Feedback loops: new-sender review queue sampled through extraction, and user signals (user manually adds the flight → flag the sender for reclassification).

12. Concepts This Problem Taught (Course Thread)

Concept	Here	Seen before / next
Prioritization under scarcity	100× gap → triage + priority tiers; SLA per tier	OS schedulers, ICU triage, ad serving

Concept	Here	Seen before / next
Cheap filter before expensive resource	Metadata triage drops 90% before any parsing	Class 4 sampling funnel; bloom filters before disk
Cache the computation at its stable level	XPaths per template, not per email	Class 4 precompute; compiled query plans
Queues as shock absorbers	P2's 1-hour SLA absorbs bursts	Class 2 Kafka ingestion buffer
Self-healing via validation	Bad output → invalidate → relearn	Read repair (Class 3), checksums
Error-cost asymmetry	False-ignore ≫ false-extract → bias recall	Fraud detection, medical screening
Capacity math must close	100K → 10K → 1K = capacity, + headroom	Class 2/4 estimation discipline